

O Cavalo Perdido

Eduardo Alcaria, Nicolas Batista
Escola Politécnica, PUCRS

24 de junho de 2026

Resumo

Este artigo apresenta a solução para o problema *O Cavalo Perdido*, proposto na disciplina de Algoritmos e Estruturas de Dados II. O problema consiste em encontrar o menor número de movimentos para um cavalo de xadrez sair de uma posição inicial e alcançar uma posição de saída em um tabuleiro toroidal com casas bloqueadas. Modela-se o tabuleiro como um grafo não-ponderado e aplica-se Busca em Largura (BFS) para encontrar o caminho mínimo de forma ótima. Os sete casos de teste fornecidos foram resolvidos com sucesso, com tempos de execução da ordem de poucos milissegundos mesmo para tabuleiros de 500×500 .

Introdução

Dentro do escopo da disciplina de Algoritmos e Estruturas de Dados II, o problema *O Cavalo Perdido* pode ser resumido da seguinte forma: dado um tabuleiro de xadrez $N \times N$ com uma posição inicial C , uma posição de saída S e casas bloqueadas marcadas com $\times$, determinar o **menor número de movimentos** que um cavalo de xadrez precisa realizar para sair de C e alcançar S .

O cavalo move-se em “L”: avança 2 casas em uma direção e 1 casa na direção perpendicular (ou vice-versa), gerando 8 deslocamentos possíveis. Além disso, o tabuleiro é *toroidal*: ao sair pela borda direita o cavalo reentra pela esquerda, e o mesmo ocorre entre topo e base, tornando o espaço de estados finito mas sem bordas para efeito de movimento.

Uma instância do problema pode não ter solução caso S seja completamente isolada por casas bloqueadas ou esteja em uma componente desconectada de C .

Nas seções a seguir apresenta-se a modelagem do problema, o algoritmo de solução e a análise dos resultados obtidos para os casos de teste.

Modelagem do Problema

O tabuleiro é modelado como um **grafo não-ponderado**, onde:

- **Vértices:** cada célula (r, c) que não seja bloqueada ($\times$) constitui um vértice. O grafo tem até N^2 vértices.
- **Arestas:** existe uma aresta entre (r_1, c_1) e (r_2, c_2) se e somente se o cavalo pode ir de um ao outro em um único movimento válido, respeitando a condição toroidal e não atingindo uma casa bloqueada no destino.

Os oito deslocamentos $(\Delta r, \Delta c)$ do cavalo são:

$$(\pm 1, \pm 2) \quad \text{e} \quad (\pm 2, \pm 1).$$

No tabuleiro toroidal, a posição resultante de um deslocamento é calculada por aritmética modular:

$$nr = ((r + \Delta r) \bmod N + N) \bmod N, \quad nc = ((c + \Delta c) \bmod N + N) \bmod N.$$

O termo $+N$ antes do segundo módulo garante que o resultado seja positivo quando $r + \Delta r < 0$ (borda superior ou esquerda).

Com essa modelagem, determinar o menor número de movimentos do cavalo é equivalente a encontrar o **menor caminho entre dois vértices em um grafo não-ponderado**, problema clássico resolvido de forma ótima pela Busca em Largura (BFS).

A complexidade desta modelagem é a seguinte:

Aspecto	Valor
Número de vértices	N^2
Grau máximo por vértice	8
Número de arestas	$O(N^2)$
Complexidade de tempo	$O(V + E) = O(N^2)$
Complexidade de espaço	$O(N^2)$

Solução

Escolha do algoritmo

A Busca em Largura (BFS) garante que a primeira vez que o destino S é encontrado, o caminho percorrido é o de **menor número de arestas** e, portanto, o menor número de movimentos. Essa garantia é válida porque o grafo é não-ponderado: todos os movimentos do cavalo têm custo uniforme igual a 1.

Outras alternativas foram consideradas:

- **DFS (Busca em Profundidade):** encontra um caminho, mas não necessariamente o menor.
- **Dijkstra:** correto para grafos com pesos, porém desnecessariamente complexo para pesos uniformes. BFS tem complexidade $O(V + E)$, enquanto Dijkstra tem $O((V + E) \log V)$.
- **A*:** eficiente quando a heurística é boa. Para tabuleiros com obstáculos aleatórios e geometria toroidal, a distância euclidiana não é heurística admissível eficaz; e como o BFS já resolve em poucos milissegundos para 500×500 , a complexidade extra não se justifica.

Algoritmo

O pseudocódigo do BFS adaptado ao problema é o seguinte:

```
1 procedimento BFS(tabuleiro, N, C, S)
2   se C = S então retornar 0
3
4   visitado[N][N] := false
5   visitado[C.r][C.c] := true
6   fila := { (C.r, C.c, distancia=0) }
7
8   enquanto fila não vazia:
9     (r, c, dist) := fila.remover_frente()
10
```

```

11     para cada (dr, dc) em MOVIMENTOS_CAVALO:
12         nr := ((r + dr) mod N + N) mod N
13         nc := ((c + dc) mod N + N) mod N
14
15         se NAO visitado[nr][nc] E tabuleiro[nr][nc] != 'x':
16             se (nr, nc) = S:
17                 retornar dist + 1
18             visitado[nr][nc] := true
19             fila.inserir_fim( (nr, nc, dist + 1) )
20
21     retornar -1    // S inacessível

```

Detalhes de implementação

A implementação foi realizada em Java. Alguns detalhes relevantes:

- Utiliza-se `ArrayDeque` em vez de `LinkedList` para a fila, reduzindo o overhead de alocação de memória e melhorando o desempenho especialmente nos tabuleiros maiores.
- A verificação de chegada ao destino é feita no momento da *inserção* na fila, não da remoção. Isso permite encerrar a busca imediatamente ao encontrar S, sem processar o restante da última camada.
- O array `visitado` também é marcado no momento da inserção, evitando que o mesmo vértice seja adicionado à fila múltiplas vezes e garantindo complexidade $O(N^2)$.

O trecho principal da implementação é apresentado a seguir:

```

1  static int[][] movimentos = {
2      {-2,-1},{-2, 1},{-1,-2},{-1, 2},
3      { 1,-2},{ 1, 2},{ 2,-1},{ 2, 1}
4  };
5
6  static int bfs(char[][] tab, int linhas, int colunas,
7      int cr, int cc, int sr, int sc) {
8      if (cr == sr && cc == sc) return 0;
9
10     boolean[][] visitado = new boolean[linhas][colunas];
11     visitado[cr][cc] = true;
12     ArrayDeque<int[]> fila = new ArrayDeque<>();
13     fila.add(new int[]{cr, cc, 0});
14
15     while (!fila.isEmpty()) {
16         int[] atual = fila.poll();
17         int r = atual[0], c = atual[1], dist = atual[2];
18
19         for (int[] m : movimentos) {
20             int nr = ((r + m[0]) % linhas + linhas) % linhas;
21             int nc = ((c + m[1]) % colunas + colunas) % colunas;
22
23             if (!visitado[nr][nc] && tab[nr][nc] != 'x') {
24                 if (nr == sr && nc == sc) return dist + 1;
25                 visitado[nr][nc] = true;

```

```

26         fila.add(new int[]{nr, nc, dist + 1});
27     }
28 }
29 }
30 return -1;
31 }

```

Validação com o exemplo do enunciado

O enunciado fornece um tabuleiro de exemplo, de dimensão 20×39 (não quadrado), e afirma que o cavalo alcança a saída em **10 pulos**. A implementação trata as dimensões de linhas e colunas de forma independente, portanto funciona para tabuleiros de qualquer dimensão $M \times N$. Ao executar o algoritmo sobre esse tabuleiro, com C na linha 4, coluna 10 e S na linha 14, coluna 34, o resultado obtido foi exatamente **10 movimentos**, reproduzindo o valor indicado no enunciado e confirmando a corretude da modelagem toroidal.

Exemplo: tabuleiro 020.txt (20×20)

O menor caso de teste é o tabuleiro 20×20 , onde o cavalo parte de $C = (19, 16)$ e deve alcançar $S = (7, 14)$.

```

linha 0  x.x.....xx....
linha 1  .....x....x
linha 2  .x....x...x...x...x
        :
        :
linha 7  xx...x.....xS...x..
        :
        :
linha 19 .....x...xxxxxxxxxC...

```

A propriedade toroidal é fundamental: a partir de $C = (19, 16)$, movimentos que sairiam pelo topo do tabuleiro reaparecem na linha 0 ou 1, ampliando significativamente a vizinhança imediata. O BFS encontra o caminho mínimo em **4 movimentos**.

Camada BFS	Situação
0	$C = (19, 16)$, posição inicial
1	Vizinhos válidos a 1 salto de C
2	Vizinhos válidos a 2 saltos de C
3	Vizinhos válidos a 3 saltos de C
4	$S = (7, 14)$ encontrado: 4 movimentos

Resultados

Os sete tabuleiros fornecidos foram processados pelo algoritmo BFS. Em todos os casos existe solução. Os resultados são apresentados na tabela a seguir.

Arquivo	Dimensão	Posição C	Posição S	Mínimo de movimentos	Tempo (ms)
020.txt	20 × 20	(19, 16)	(7, 14)	4	<1
060.txt	60 × 60	(24, 48)	(49, 6)	15	1
120.txt	120 × 120	(115, 65)	(16, 82)	14	1
200.txt	200 × 200	(109, 91)	(161, 188)	51	5
300.txt	300 × 300	(103, 136)	(22, 15)	68	6
400.txt	400 × 400	(242, 164)	(190, 66)	52	2
500.txt	500 × 500	(347, 282)	(354, 36)	123	14

Observa-se que o número de movimentos não cresce monotonicamente com o tamanho do tabuleiro. O caso 120 × 120 exige menos movimentos (14) do que o 60 × 60 (15), e o 400 × 400 (52) menos do que o 200 × 200 (51). Isso ocorre porque a dificuldade de cada instância depende da posição relativa de C e S e da distribuição dos obstáculos, que são independentes em cada tabuleiro.

O tempo de execução permaneceu na ordem de poucos milissegundos mesmo para o maior tabuleiro (500 × 500, com 250 000 células), confirmando a eficiência prática do BFS com complexidade $O(N^2)$. Os tempos da tabela são de uma execução representativa e variam conforme a máquina.

Conclusões

O problema *O Cavalo Perdido* foi modelado como busca pelo menor caminho em um grafo não-ponderado sobre um tabuleiro toroidal com obstáculos. A escolha de BFS mostrou-se adequada e eficiente:

- **Otimidade:** BFS garante o menor número de movimentos em grafos não-ponderados.
- **Eficiência:** complexidade $O(N^2)$ em tempo e espaço, com todos os casos resolvidos em poucos milissegundos.
- **Corretude no tabuleiro toroidal:** a aritmética modular garante que os movimentos que cruzam bordas são tratados corretamente.
- **Impossibilidade:** quando não existe caminho, o BFS encerra após visitar todos os vértices acessíveis a partir de C e retorna -1 .

Em todos os sete casos de teste o cavalo conseguiu alcançar a saída. O caso mais simples exigiu 4 movimentos (tabuleiro 20 × 20) e o mais complexo 123 movimentos (tabuleiro 500 × 500).

Uma possível extensão seria explorar heurísticas admissíveis para A*, o que poderia reduzir o número de estados explorados em instâncias com solução de muitos passos. No entanto, dado o desempenho já obtido com BFS puro, tal refinamento não se mostrou necessário para os casos de teste fornecidos.

Referências

- [1] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C.: “**Introduction to Algorithms**”. 3. ed. MIT Press, Cambridge, 2009.
- [2] Sedgewick, R.; Wayne, K.: “**Algorithms**”. 4. ed. Addison-Wesley, 2011.
- [3] Skiena, S. S.: “**The Algorithm Design Manual**”. 2. ed. Springer, 2008.